Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Quiz 2 – SQL Avanzado en Oracle — GUÍA DE REVISIÓN AUTÓNOMA

Para el estudiante. Esta guía explica, paso a paso, la solución completa del Quiz 2 sobre el esquema de la aerolínea AeroAndes. No es una hoja de respuestas: cada afirmación va acompañada de la consulta que la comprueba y del resultado que debe obtenerse. La intención es que puedas montar el esquema, ejecutar cada experimento y comprobar por ti mismo lo que aquí se afirma, en lugar de creerlo.

Cómo usar esta guía

- Monta el esquema con los dos scripts que acompañan a esta guía: primero el de creación de tablas y después el de carga de datos. Cualquier instancia de Oracle 19c sirve, incluida la edición gratuita.
- Lee cada sección en orden. Antes de leer la explicación, ejecuta la consulta que la encabeza e intenta predecir el resultado. La predicción fallida es la que enseña.
- Todos los números que aparecen en esta guía provienen de los datos cargados por el script. Si al ejecutar obtienes valores distintos, la carga quedó incompleta.
- Al final hay una sección de autoevaluación. Si puedes responderla sin volver atrás, dominaste el tema.

Los datos con los que vas a trabajar

```
aeropuerto     7 filas        vuelo         37 filas
aeronave       3 filas        incidencia    34 filas
pasajero      40 filas        reserva      140 filas

Periodo: enero a abril de 2026.
Rutas operadas: BOG-MDE, BOG-CTG, BOG-CLO, BOG-BAQ, MDE-CTG, CLO-SMR.
```

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

PARTE 0 — LA IDEA QUE ATRAVIESA TODO EL QUIZ

Cuatro de los cinco defectos de la Pregunta 1 y las tres partes de la Pregunta 2 se explican con una sola idea: el orden en que Oracle evalúa las cláusulas de una sentencia no es el orden en que las escribimos.

Escribimos así:

```
SELECT   ...        <- 6.o
FROM     ...        <- 1.o
WHERE    ...        <- 2.o
GROUP BY ...        <- 3.o
HAVING   ...        <- 4.o
ORDER BY ...        <- 7.o
```

Oracle evalúa así:

```
  1.  FROM y JOIN      Se arma el conjunto de filas combinando las tablas.
                       Aqui es donde una fila puede multiplicarse.

  2.  WHERE            Se descartan filas individuales. Todavia no existe
                       ninguna agrupacion, ningun total y ningun ranking.

  3.  GROUP BY         Las filas que sobreviven se colapsan en grupos.
                       A partir de aqui una fila = un grupo.

  4.  HAVING           Se descartan grupos completos usando agregados.

  5.  Funciones        RANK, LAG, SUM() OVER, ROW_NUMBER... Reciben como
      analiticas       entrada el resultado del paso 4. Por eso PUEDEN
                       operar sobre SUM(), y por eso NO existen todavia
                       cuando se evalua el WHERE.

  6.  SELECT           Se arma la lista de columnas y se aplican los alias.

  7.  ORDER BY         Se ordena el resultado final. Es lo ultimo.
```

Las tres consecuencias que debes tener siempre presentes
Primera. Una función analítica no puede usarse en el WHERE, porque en ese momento todavía no ha sido calculada. Si necesitas filtrar por un ranking, tienes que calcularlo en un nivel y filtrarlo en otro. Esa es la razón de ser de la cláusula WITH en este quiz.
Segunda. Un alias definido en el SELECT no existe en el WHERE ni en el GROUP BY, porque el SELECT se evalúa después. Sí existe en el ORDER BY, que va todavía más tarde.
Tercera. ROWNUM se asigna cuando la fila supera el WHERE, es decir en el paso 2. No numera el resultado final: numera el flujo de filas crudas. Es la fuente del segundo defecto de la Pregunta 1.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

PARTE 1 — PREGUNTA 1: DIAGNÓSTICO Y REESCRITURA

El requerimiento y la consulta entregada

El requerimiento era: las cinco rutas con mayor ingreso desde el 1 de enero de 2026, excluyendo del cálculo a los pasajeros que hayan referido a otro pasajero. La consulta entregada fue:

```
SELECT  v.cod_origen || '-' || v.cod_destino          AS ruta,
        v.num_vuelo                                   AS numero,
        COUNT(*)                                      AS incidencias,
        SUM(r.tarifa)                                 AS ingreso,
        RANK() OVER (ORDER BY SUM(r.tarifa) DESC)     AS puesto
FROM      vuelo v
JOIN      reserva r      ON r.id_vuelo = v.id_vuelo
LEFT JOIN incidencia i   ON i.id_vuelo = v.id_vuelo
WHERE   v.fecha_salida >= DATE '2026-01-01'
  AND   ROWNUM <= 5
  AND   r.id_pasajero NOT IN (SELECT p.id_pasajero_referido
                              FROM   pasajero p)
GROUP BY v.cod_origen, v.cod_destino
ORDER BY ingreso DESC;
```

Tiene cinco defectos. Vamos uno por uno. Para cada uno verás qué ocurre, cómo comprobarlo, por qué ocurre y cómo se corrige.

Defecto 1 — Una columna del SELECT no está en el GROUP BY

Qué ocurre. La sentencia ni siquiera llega a ejecutarse. Oracle responde:

```
ORA-00979: not a GROUP BY expression
```

Por qué. El GROUP BY colapsa las filas en grupos: un grupo por cada combinación de cod_origen y cod_destino. Después de ese colapso, cada fila del resultado representa a una ruta completa, que puede abarcar diez vuelos distintos. La columna v.num_vuelo pide el número de vuelo de esa fila, pero la fila ya no corresponde a un vuelo: corresponde a una ruta. Oracle no tiene forma de escoger cuál de los diez números mostrar, y en lugar de inventarse uno, rechaza la sentencia.

La regla general. Toda columna que aparezca en el SELECT debe cumplir una de dos condiciones: o está en el GROUP BY, o está dentro de una función de agregado como SUM, COUNT, MIN o MAX. No hay tercera opción.

Corrección. Eliminar num_vuelo del SELECT. El requerimiento pide un informe por ruta, no por vuelo, de modo que esa columna no debería estar ahí. Si de verdad se quisiera conservar algún número de vuelo, habría que decidir cuál, por ejemplo con MIN(v.num_vuelo), pero eso ya sería otro informe.

Detalle importante para tu diagnóstico

Este es el defecto que impide la ejecución. Mientras no lo corrijas, los otros cuatro permanecen ocultos: no puedes observarlos porque la sentencia nunca llega a producir un resultado. Si en tu respuesta escribiste que la consulta «funciona pero devuelve mal el ingreso», ese es un error de razonamiento, no un descuido: la consulta no funciona en absoluto.

Defecto 2 — ROWNUM en el WHERE

Qué ocurre. La consulta se ejecuta y devuelve un resultado que parece razonable, pero no son las cinco rutas de mayor ingreso. Normalmente devuelve una sola fila.

Compruébalo. Ejecuta esta versión reducida:

```
SELECT  v.cod_origen || '-' || v.cod_destino AS ruta,
        COUNT(*)      AS filas_crudas,
        SUM(r.tarifa) AS ingreso
FROM    vuelo v
JOIN    reserva r ON r.id_vuelo = v.id_vuelo
```

```
WHERE   v.fecha_salida >= DATE '2026-01-01'
  AND   ROWNUM <= 5
GROUP BY v.cod_origen, v.cod_destino;
```

Obtendrás una sola fila, con COUNT(*) = 5. Cinco filas crudas, agrupadas, de una única ruta.

Por qué. Vuelve al orden de evaluación. ROWNUM se asigna en el paso 2, a medida que las filas van superando el filtro del WHERE. En ese momento no hay agrupación, no hay ingreso calculado y no hay ordenamiento. La condición ROWNUM <= 5 corta el flujo apenas pasan cinco filas del join, y esas cinco filas son las que el plan de ejecución produzca primero, en un orden que ni siquiera está garantizado entre ejecuciones. Después se agrupan esas cinco y se ordenan. El ORDER BY llega cuando ya solo queda una fila que ordenar.

Una analogía. Es como si para escoger a los cinco mejores estudiantes de un curso tomaras a los cinco primeros que entran al salón y calcularas el promedio solo entre ellos. El criterio de selección se aplicó antes de conocer las notas.

Corrección. Calcular primero la agregación y el ranking, y filtrar después en un nivel externo:

```
WITH agregado AS (
  SELECT ... SUM(r.tarifa) AS ingreso ... GROUP BY ...
),
clasificado AS (
  SELECT ruta, ingreso,
         RANK() OVER (ORDER BY ingreso DESC) AS puesto
  FROM   agregado
)
SELECT * FROM clasificado WHERE puesto <= 5;
```

Alternativa válida. Desde Oracle 12c existe FETCH FIRST 5 ROWS ONLY, que sí se evalúa después del ORDER BY y por lo tanto sí devuelve las cinco primeras del resultado ordenado. Es correcta, pero fíjate en la diferencia conceptual: el problema del original no es «usar ROWNUM», sino el momento en que ROWNUM se evalúa.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.



Defecto 3 — NOT IN sobre una columna que admite nulos

Qué ocurre. La consulta se ejecuta sin ningún error y devuelve el conjunto vacío. Cero filas. Este es, con diferencia, el defecto más peligroso de los cinco, porque no hay nada que avise de que algo salió mal.

Compruébalo. Ejecuta las tres consultas siguientes en orden:

```
-- 1. Cuantas reservas sobreviven al filtro tal como esta escrito
SELECT COUNT(*) AS filas
FROM   reserva r
WHERE  r.id_pasajero NOT IN (SELECT p.id_pasajero_referido FROM pasajero p);
-- Resultado: 0

-- 2. Que devuelve realmente la subconsulta
SELECT COUNT(*) AS total,
       COUNT(id_pasajero_referido) AS no_nulos,
       COUNT(*) - COUNT(id_pasajero_referido) AS nulos
FROM   pasajero;
-- Resultado: total 40, no_nulos 6, nulos 34

-- 3. La version correcta, con NOT EXISTS
SELECT COUNT(*) AS filas
FROM   reserva r
WHERE  NOT EXISTS (SELECT 1 FROM pasajero p
                   WHERE  p.id_pasajero_referido = r.id_pasajero);
-- Resultado: 131 de las 140 reservas
```

Por qué. Hay que desarmar la expresión. Cuando escribes x NOT IN (a, b, c), Oracle la evalúa como:

```
NOT ( x = a  OR  x = b  OR  x = c )
```

Ahora supón que uno de esos valores es nulo. La comparación x = NULL no devuelve verdadero ni falso: devuelve UNKNOWN, porque nulo significa «no se sabe» y no se puede saber si algo es igual a lo que no se conoce. Entonces la expresión queda:

```
NOT ( FALSE  OR  FALSE  OR  UNKNOWN )   =   NOT ( UNKNOWN )   =   UNKNOWN
```

Y el WHERE solo deja pasar las filas cuya condición sea verdadera. UNKNOWN no es verdadero, así que ninguna fila pasa. Como en la tabla PASAJERO hay 34 registros con la columna de referido en nulo, la subconsulta devuelve 34 nulos y el resultado es siempre vacío, para toda reserva, sin excepción.

Compáralo con IN, que se comporta al revés

Con IN el nulo es inofensivo: x IN (a, b, NULL) se evalúa como (x = a OR x = b OR UNKNOWN). Si x coincide con a, la expresión da TRUE y la fila pasa, porque en una disyunción basta un verdadero.

Con NOT IN el nulo es letal, porque la negación convierte esa disyunción en una conjunción: se exige que x sea distinto de TODOS los valores, y no se puede afirmar que x sea distinto de un valor desconocido.

Regla práctica: antes de escribir NOT IN, pregúntate si la columna de la subconsulta admite nulos. Si la respuesta es sí o no estás seguro, usa NOT EXISTS.

Las dos correcciones. La primera es NOT EXISTS con la correlación explícita, que es la forma preferida porque maneja los nulos correctamente por diseño. La segunda es añadir un filtro a la subconsulta:

```
-- Opcion A (recomendada)
AND NOT EXISTS (SELECT 1 FROM pasajero p
                WHERE  p.id_pasajero_referido = r.id_pasajero)

-- Opcion B
AND r.id_pasajero NOT IN (SELECT p.id_pasajero_referido
                          FROM   pasajero p
                          WHERE  p.id_pasajero_referido IS NOT NULL)
```

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

En los datos cargados los pasajeros que refirieron a alguien son tres: los identificadores 3, 11 y 25. Entre los tres tienen 6 reservas por 1.900.000 en total, todas en la ruta CLO-SMR. Ese es el monto que la exclusión debe retirar del cálculo.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Defecto 4 — El join a INCIDENCIA multiplica las filas de RESERVA

Qué ocurre. El ingreso de cada ruta aparece inflado, y la columna rotulada «incidencias» no cuenta incidencias sino filas del join. Los dos valores están mal a la vez.

El mecanismo. VUELO se une a RESERVA y, en la misma sentencia, a INCIDENCIA. Las dos son relaciones de uno a muchos con VUELO. El resultado no es la suma de las dos ramas: es el producto.

```
Un vuelo con 3 reservas y 2 incidencias produce 3 x 2 = 6 filas:

   reserva A  -  incidencia 1        La tarifa de la reserva A
   reserva A  -  incidencia 2        aparece DOS veces.
   reserva B  -  incidencia 1
   reserva B  -  incidencia 2        Lo mismo con B y con C.
   reserva C  -  incidencia 1
   reserva C  -  incidencia 2

SUM(r.tarifa) suma seis tarifas donde solo hay tres reservas.
COUNT(*) devuelve 6 donde solo hay 2 incidencias.
```

Compruébalo. Esta consulta pone las dos cifras lado a lado:

```
WITH correcto AS (
  SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
         SUM(r.tarifa) AS ingreso_correcto
  FROM   vuelo v
  JOIN   reserva r ON r.id_vuelo = v.id_vuelo
  WHERE  r.estado <> 'ANULADA'
  GROUP BY v.cod_origen, v.cod_destino
),
inflado AS (
  SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
         SUM(r.tarifa) AS ingreso_inflado
  FROM      vuelo v
  JOIN      reserva r    ON r.id_vuelo = v.id_vuelo
  LEFT JOIN incidencia i ON i.id_vuelo = v.id_vuelo
  WHERE  r.estado <> 'ANULADA'
  GROUP BY v.cod_origen, v.cod_destino
)
SELECT c.ruta, c.ingreso_correcto, f.ingreso_inflado,
       ROUND(f.ingreso_inflado / c.ingreso_correcto, 2) AS factor
FROM   correcto c JOIN inflado f ON f.ruta = c.ruta
ORDER BY c.ingreso_correcto DESC;
```

Resultado sobre los datos cargados:

```
RUTA      INGRESO_CORRECTO   INGRESO_INFLADO   FACTOR
BOG-MDE         15.700.000        21.500.000     1,37
BOG-CTG         14.000.000        20.400.000     1,46
BOG-CLO         11.300.000        15.900.000     1,41
BOG-BAQ         10.100.000        14.300.000     1,42
MDE-CTG          5.100.000         5.100.000     1,00
CLO-SMR          4.200.000         6.000.000     1,43
```

Fíjate en la fila de MDE-CTG

Su factor es exactamente 1,00: el ingreso inflado coincide con el correcto. La razón es que esa ruta no tiene ninguna incidencia registrada, de modo que el LEFT JOIN no multiplica nada.

Esto es importante para entender el defecto: no aparece siempre ni de forma uniforme. Depende de cuántas filas tenga el lado múltiple en cada caso. Una consulta con este error puede parecer correcta durante meses y desviarse justo cuando los datos cambian.

Un agravante que conviene notar. El join a DEVOLUCION se hace únicamente por id_vuelo. Fíjate en el esquema: INCIDENCIA se relaciona con VUELO, así que aquí la relación es correcta. Pero verifica siempre que la condición del join use la clave

completa; en modelos donde la clave foránea es compuesta, unir por una sola columna empareja filas que no se corresponden y produce el mismo tipo de multiplicación.

Corrección. Agregar cada rama por separado, en cláusulas WITH independientes, y unir los resultados ya agregados. Cuando cada CTE devuelve una fila por ruta, unirlas no puede multiplicar nada.

Defecto 5 — No se excluyen las reservas anuladas

Qué ocurre. El ingreso incluye dinero que nunca se percibió. Las reglas de negocio del enunciado son explícitas: el campo estado toma los valores CONFIRMADA, VOLADA o ANULADA, y las reservas anuladas no generan ingreso.

Compruébalo.

```
SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
       SUM(CASE WHEN r.estado <> 'ANULADA' THEN r.tarifa ELSE 0 END) AS correcto,
       SUM(r.tarifa)                                                 AS con_anuladas,
       SUM(CASE WHEN r.estado  = 'ANULADA' THEN r.tarifa ELSE 0 END) AS exceso
FROM   vuelo v
JOIN   reserva r ON r.id_vuelo = v.id_vuelo
GROUP BY v.cod_origen, v.cod_destino
ORDER BY correcto DESC;
```

```
RUTA        CORRECTO   CON_ANULADAS      EXCESO
BOG-MDE   15.700.000     18.840.000   3.140.000
BOG-CTG   14.000.000     16.800.000   2.800.000
BOG-CLO   11.300.000     13.470.000   2.170.000
BOG-BAQ   10.100.000     11.920.000   1.820.000
MDE-CTG    5.100.000      5.880.000     780.000
CLO-SMR    4.200.000      4.910.000     710.000
```

Este defecto es distinto de los anteriores. Los cuatro primeros son errores técnicos: se detectan leyendo la consulta y conociendo el motor. Este solo se detecta leyendo las reglas de negocio del enunciado. Una consulta puede ser impecable desde el punto de vista del lenguaje y responder a una pregunta distinta de la que se hizo. Acostúmbrate a releer el requerimiento después de escribir la consulta.

Corrección. Añadir r.estado <> 'ANULADA' al WHERE. Y un detalle de rigor: si la columna estado admitiera nulos, ese predicado descartaría también las filas con estado nulo, porque NULL <> 'ANULADA' se evalúa como UNKNOWN. En ese caso habría que escribir NVL(r.estado, 'SIN_ESTADO') <> 'ANULADA'. Es el mismo mecanismo del defecto 3, apareciendo en otro lugar.

Un elemento que parece defecto y no lo es

Muchos estudiantes señalan esta línea como error:

```
RANK() OVER (ORDER BY SUM(r.tarifa) DESC) AS puesto
```

«Una función analítica no puede recibir una función de agregado», dicen. Es falso. Vuelve al orden de evaluación: las funciones analíticas se calculan en el paso 5, después del GROUP BY del paso 3. Cuando RANK se evalúa, el SUM ya está calculado y es simplemente un valor más de la fila. Aquí hay dos funciones distintas trabajando en momentos distintos: el SUM de adentro es de agregado y el RANK de afuera es analítico.

Esa construcción es válida y frecuente en Oracle. Señalarla como defecto no suma puntos y revela justamente el malentendido que el quiz busca corregir.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Literal b — La reescritura, construida por partes

En lugar de mostrarte la consulta final, vamos a construirla. Ejecuta cada paso y observa cómo cambia el resultado.

Paso 1 — El ingreso por ruta, sin ningún filtro

```
SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
       SUM(r.tarifa) AS ingreso
FROM   vuelo v
JOIN   reserva r ON r.id_vuelo = v.id_vuelo
WHERE  v.fecha_salida >= DATE '2026-01-01'
GROUP BY v.cod_origen, v.cod_destino
ORDER BY ingreso DESC;
```

Observa que aquí no aparece INCIDENCIA. Al no estar, no hay nada que multiplique las filas. Resultado:

```
BOG-MDE  18.840.000     BOG-BAQ  11.920.000
BOG-CTG  16.800.000     MDE-CTG   5.880.000
BOG-CLO  13.470.000     CLO-SMR   4.910.000
```

Paso 2 — Excluir las reservas anuladas

```
  AND  r.estado <> 'ANULADA'
```

El ingreso baja en todas las rutas. Este es el efecto del defecto 5:

```
BOG-MDE  15.700.000     BOG-BAQ  10.100.000
BOG-CTG  14.000.000     MDE-CTG   5.100.000
BOG-CLO  11.300.000     CLO-SMR   4.200.000
```

Paso 3 — Excluir a los pasajeros que refirieron a alguien

```
  AND  NOT EXISTS (SELECT 1
                   FROM   pasajero p
                   WHERE  p.id_pasajero_referido = r.id_pasajero)
```

Ahora fíjate en algo revelador: solo cambia una ruta.

```
BOG-MDE  15.700.000     BOG-BAQ  10.100.000
BOG-CTG  14.000.000     MDE-CTG   5.100.000
BOG-CLO  11.300.000     CLO-SMR   2.300.000   <- baja 1.900.000
```

Los tres pasajeros referidores tienen todas sus reservas en CLO-SMR, de modo que el filtro solo afecta a esa ruta. Compara este resultado con el del paso anterior: si hubieras usado NOT IN en lugar de NOT EXISTS, el resultado no habría sido este sino ninguna fila.

Paso 4 — Contar los vuelos con incidencia, en su propia consulta

```
SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
       COUNT(DISTINCT i.id_vuelo) AS vuelos_con_incidencia
FROM   vuelo v
JOIN   incidencia i ON i.id_vuelo = v.id_vuelo
WHERE  v.fecha_salida >= DATE '2026-01-01'
GROUP BY v.cod_origen, v.cod_destino;
```

El enunciado pide «el número de vuelos distintos que registraron al menos una incidencia». Cada palabra importa. Como un vuelo puede tener varias incidencias, hay que contar vuelos distintos y no incidencias, de ahí el DISTINCT. Y el join es interno, no externo, porque solo interesan los vuelos que sí tuvieron incidencia.

```
BOG-MDE  4     BOG-BAQ  4
BOG-CTG  4     MDE-CTG  1
```

```
BOG-CLO  3     CLO-SMR  3
```

Paso 5 — Unir las dos ramas y clasificar

Ya tenemos dos consultas, cada una con una fila por ruta. Ahora sí podemos unirlas sin riesgo de multiplicación, porque ninguna de las dos tiene filas repetidas por ruta. La consulta completa queda así:

```
WITH ingresos AS (
  SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
         SUM(r.tarifa)                       AS ingreso
  FROM   vuelo v
  JOIN   reserva r ON r.id_vuelo = v.id_vuelo
  WHERE  v.fecha_salida >= DATE '2026-01-01'
    AND  r.estado <> 'ANULADA'
    AND  NOT EXISTS (SELECT 1
                     FROM   pasajero p
                     WHERE  p.id_pasajero_referido = r.id_pasajero)
  GROUP BY v.cod_origen, v.cod_destino
),
incidentes AS (
  SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
         COUNT(DISTINCT i.id_vuelo)           AS vuelos_con_incidencia
  FROM   vuelo v
  JOIN   incidencia i ON i.id_vuelo = v.id_vuelo
  WHERE  v.fecha_salida >= DATE '2026-01-01'
  GROUP BY v.cod_origen, v.cod_destino
),
ranking AS (
  SELECT g.ruta,
         g.ingreso,
         NVL(x.vuelos_con_incidencia, 0)       AS vuelos_con_incidencia,
         RANK() OVER (ORDER BY g.ingreso DESC) AS puesto
  FROM   ingresos g
  LEFT JOIN incidentes x ON x.ruta = g.ruta
)
SELECT ruta, ingreso, vuelos_con_incidencia, puesto
FROM   ranking
WHERE  puesto <= 5
ORDER BY puesto;
```

Resultado final:

```
RUTA        INGRESO   VUELOS_CON_INCIDENCIA   PUESTO
BOG-MDE  15.700.000              4               1
BOG-CTG  14.000.000              4               2
BOG-CLO  11.300.000              3               3
BOG-BAQ  10.100.000              4               4
MDE-CTG   5.100.000              1               5
```

CLO-SMR queda fuera del top cinco, que es exactamente lo que el filtro debe hacer.

Tres decisiones de diseño que debes poder justificar
El LEFT JOIN entre las dos CTE, y no un join interno: si una ruta no tuviera ninguna incidencia, un join interno la haría desaparecer del informe, cuando debería aparecer con cero. El NVL convierte ese nulo en cero.
El filtro del puesto en la consulta externa, y no dentro de la CTE que lo calcula: es la consecuencia directa de que las funciones analíticas se evalúan después del WHERE.
Los dos filtros de negocio aplicados dentro de la CTE de ingresos y no al final: si se aplicaran después de agregar, ya sería tarde, porque las tarifas anuladas y las de los referidores ya estarían sumadas.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

PARTE 2 — PREGUNTA 2: CONSTRUCCIÓN Y ANÁLISIS

Literal a — Ranking mensual por ruta

Se pide, para cada mes de 2026, las tres rutas de mayor ingreso de ese mes, con desempate determinístico por código de ruta.

Paso 1 — Agrupar por mes y por ruta

```
SELECT TRUNC(v.fecha_salida, 'MM')          AS mes,
       v.cod_origen || '-' || v.cod_destino AS ruta,
       SUM(r.tarifa)                        AS ingreso
FROM   vuelo v
JOIN   reserva r ON r.id_vuelo = v.id_vuelo
WHERE  v.fecha_salida >= DATE '2026-01-01'
  AND  v.fecha_salida <  DATE '2027-01-01'
  AND  r.estado <> 'ANULADA'
GROUP BY TRUNC(v.fecha_salida, 'MM'),
         v.cod_origen || '-' || v.cod_destino;
```

Por qué TRUNC y no TO_CHAR. TRUNC(fecha, 'MM') lleva la fecha al primer día de su mes y conserva el tipo DATE, de modo que sigue ordenándose cronológicamente y sigue siendo comparable. TO_CHAR(fecha, 'MM') devolvería la cadena '01' para enero de 2026 y también para enero de 2027, mezclando años. Aquí el rango está acotado a un solo año, pero es una costumbre que conviene adquirir bien desde el principio.

Por qué el rango de fechas y no EXTRACT(YEAR ...). Ambos dan el mismo resultado, pero aplicar una función sobre la columna filtrada impide que el optimizador use un índice sobre fecha_salida. Con dos comparaciones de rango, el índice sí puede aprovecharse.

Paso 2 — Clasificar dentro de cada mes

```
RANK() OVER (PARTITION BY mes
             ORDER BY ingreso DESC, ruta ASC) AS puesto
```

El PARTITION BY es lo esencial. Sin él, el ranking sería global sobre todos los meses juntos: el primer puesto se lo llevaría el mejor mes del año y los meses flojos no tendrían representantes. Con la partición, el conteo se reinicia en cada mes y cada uno tiene su propio primero, segundo y tercero.

El segundo criterio del ORDER BY es lo que pide el enunciado. «Si dos rutas empatan en ingreso, el desempate debe ser determinístico: gana la ruta cuyo código sea menor alfabéticamente.» Eso se traduce en añadir ruta ASC después de ingreso DESC.

Paso 3 — Filtrar y ordenar

```
WITH ingreso_mes AS ( ... paso 1 ... ),
ordenado AS (
  SELECT mes, ruta, ingreso,
         RANK() OVER (PARTITION BY mes
                      ORDER BY ingreso DESC, ruta ASC) AS puesto
  FROM   ingreso_mes
)
SELECT TO_CHAR(mes, 'YYYY-MM') AS mes, ruta, ingreso, puesto
FROM   ordenado
WHERE  puesto <= 3
ORDER BY mes, puesto;
```

Resultado sobre los datos cargados:

```
MES      RUTA        INGRESO   PUESTO
2026-01  BOG-MDE   5.000.000     1
2026-01  BOG-CTG   4.000.000     2
2026-01  BOG-CLO   3.000.000     3
2026-02  BOG-MDE   4.800.000     1
2026-02  BOG-BAQ   4.000.000     2    <- empate resuelto por alfabetico
```

```
2026-02  BOG-CTG   4.000.000     3
2026-03  BOG-BAQ   3.600.000     1    <- empate resuelto por alfabetico
2026-03  BOG-CLO   3.600.000     2
2026-03  BOG-MDE   3.000.000     3
2026-04  BOG-CTG   3.500.000     1
2026-04  BOG-MDE   2.900.000     2
2026-04  BOG-CLO   1.200.000     3    <- empate con MDE-CTG, gana BOG-CLO
```

Experimento que vale la pena hacer

Ejecuta la consulta quitando el criterio ruta ASC del ORDER BY de la ventana y compara. En febrero, BOG-BAQ y BOG-CTG tienen ambos 4.000.000; sin el desempate los dos reciben el puesto 2, ninguno recibe el 3 y el resultado deja de tener exactamente tres filas por mes.

En abril el efecto es aún más visible: BOG-CLO y MDE-CTG empatan en 1.200.000 y ambos entrarían al top tres, devolviendo cuatro rutas donde se pidieron tres.

La justificación que pedía el enunciado. ¿Por qué el filtro del puesto no puede escribirse en el WHERE de la misma consulta donde se calcula? Porque el WHERE se evalúa en el paso 2 y las funciones analíticas en el paso 5. En el momento del WHERE la columna puesto todavía no existe. Si lo intentas, Oracle responde ORA-30483 si escribes la función completa, o ORA-00904 si intentas usar el alias.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Literal b — El acumulado que no acumula

La consulta del compañero pretendía obtener el ingreso acumulado reserva por reserva dentro de cada aeropuerto de origen:

```
SUM(r.tarifa) OVER (PARTITION BY v.cod_origen
                    ORDER BY r.fecha_compra) AS acumulado
```

Y observó que las reservas compradas el mismo día muestran todas el mismo valor, sin ningún error.

Compruébalo primero

```
SELECT r.id_reserva, r.fecha_compra, r.tarifa,
       SUM(r.tarifa) OVER (PARTITION BY v.cod_origen
                           ORDER BY r.fecha_compra) AS acum_range,
       SUM(r.tarifa) OVER (PARTITION BY v.cod_origen
                           ORDER BY r.fecha_compra, r.id_reserva
                           ROWS BETWEEN UNBOUNDED PRECEDING
                                    AND CURRENT ROW)  AS acum_rows
FROM   reserva r
JOIN   vuelo v ON v.id_vuelo = r.id_vuelo
WHERE  v.cod_origen = 'BOG'
  AND  r.fecha_compra = DATE '2026-01-08'
ORDER BY r.id_reserva;
```

En el aeropuerto BOG hay 17 reservas compradas el 8 de enero de 2026. Este es el resultado, y es la imagen que debes quedarte de esta pregunta:

```
ID_RESERVA  FECHA_COMPRA    TARIFA    ACUM_RANGE    ACUM_ROWS
      5001    2026-01-08 1.000.000    10.080.000    1.000.000
      5002    2026-01-08 1.000.000    10.080.000    2.000.000
      5003    2026-01-08 1.000.000    10.080.000    3.000.000
      5004    2026-01-08   500.000    10.080.000    3.500.000
      5029    2026-01-08   800.000    10.080.000    4.300.000
      5030    2026-01-08   800.000    10.080.000    5.100.000
      5031    2026-01-08   800.000    10.080.000    5.900.000
      5032    2026-01-08   400.000    10.080.000    6.300.000
      5057    2026-01-08   600.000    10.080.000    6.900.000
      5058    2026-01-08   600.000    10.080.000    7.500.000
      5059    2026-01-08   600.000    10.080.000    8.100.000
      5060    2026-01-08   300.000    10.080.000    8.400.000
      5083    2026-01-08   370.000    10.080.000    8.770.000
      5084    2026-01-08   370.000    10.080.000    9.140.000
      5085    2026-01-08   370.000    10.080.000    9.510.000
      5086    2026-01-08   390.000    10.080.000    9.900.000
      5087    2026-01-08   180.000    10.080.000   10.080.000
```

La columna de la izquierda repite diecisiete veces el mismo número. La de la derecha avanza fila por fila hasta llegar, en la última, al mismo valor. Ese es exactamente el punto.

Por qué ocurre

Cuando escribes ORDER BY dentro de la cláusula OVER y no especificas un marco de ventana, Oracle no deja el marco vacío: aplica uno por defecto, y ese marco es

```
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

La palabra decisiva es RANGE. Un marco RANGE no razona en términos de posiciones de filas sino de valores de la expresión del ORDER BY. Para cada fila, incluye en la ventana a todas las filas anteriores más todas aquellas que tengan el mismo valor de ordenamiento, sin importar en qué posición estén.

```
Con RANGE, para la fila 5001 la ventana contiene:

   todas las filas con fecha_compra < 2026-01-08   -> ninguna
 + todas las filas con fecha_compra = 2026-01-08   -> las 17
   ------------------------------------------------------------
   17 filas   ->   suma 10.080.000
```

```
Y para la fila 5087, la ultima de ese dia, la ventana contiene
exactamente las mismas 17 filas. Por eso el valor no cambia.

Con ROWS, para la fila 5001 la ventana contiene:

   desde la primera fila de la particion hasta la fila actual
   ------------------------------------------------------------
   1 fila     ->   suma 1.000.000
```

El término técnico. Las filas que comparten el mismo valor en la expresión del ORDER BY se llaman «pares» o peers. Un marco RANGE trata a todos los pares como una unidad indivisible: o entran todos a la ventana, o no entra ninguno. Un marco ROWS los distingue por su posición.

La corrección, y el detalle que casi nadie menciona

```
SUM(r.tarifa) OVER (PARTITION BY v.cod_origen
                    ORDER BY r.fecha_compra, r.id_reserva
                    ROWS BETWEEN UNBOUNDED PRECEDING
                             AND CURRENT ROW) AS acumulado
```

Cambiar RANGE por ROWS resuelve el problema, pero introduce uno nuevo. Con ROWS el resultado pasa a depender del orden en que queden las filas empatadas, y ese orden no está garantizado: dos ejecuciones podrían devolver acumulados distintos para las mismas filas. Por eso hay que añadir un criterio de desempate al ORDER BY de la ventana, en este caso id_reserva, que al ser clave primaria garantiza un orden único.

El error de diagnóstico más frecuente en esta pregunta
Atribuir el problema a que «falta PARTITION BY» o a que «debería haber un GROUP BY». Ninguna de las dos cosas es cierta: la partición está correctamente escrita y un GROUP BY colapsaría las filas, que es justo lo contrario de lo que se busca. El problema no está en qué filas entran a la partición, sino en cuáles entran a la ventana de cada fila dentro de esa partición. Regla práctica para llevarte: si usas una función de ventana acumulativa y hay cualquier posibilidad de valores repetidos en el ORDER BY, escribe el marco ROWS de forma explícita. No confíes en el valor por defecto.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

Literal c — Detección de caídas mensuales

Se piden las rutas y meses de 2026 en los que el ingreso cayó más de un 30 % respecto del mes anterior de esa misma ruta.

La herramienta: LAG

LAG permite que una fila consulte el valor de una fila anterior dentro de su partición. Es exactamente lo que hace falta aquí: comparar cada mes con el que le precede.

```
LAG(ingreso) OVER (PARTITION BY ruta ORDER BY mes) AS ingreso_anterior

  PARTITION BY ruta   la comparacion se reinicia en cada ruta, de modo
                      que el enero de BOG-MDE no se compara con el abril
                      de BOG-CTG.

  ORDER BY mes        define que significa 'anterior' dentro de cada ruta.
```

La consulta completa

```
WITH ingreso_mes AS (
  SELECT v.cod_origen || '-' || v.cod_destino AS ruta,
         TRUNC(v.fecha_salida, 'MM')          AS mes,
         SUM(r.tarifa)                        AS ingreso
  FROM   vuelo v
  JOIN   reserva r ON r.id_vuelo = v.id_vuelo
  WHERE  v.fecha_salida >= DATE '2026-01-01'
    AND  v.fecha_salida <  DATE '2027-01-01'
    AND  r.estado <> 'ANULADA'
  GROUP BY v.cod_origen || '-' || v.cod_destino,
           TRUNC(v.fecha_salida, 'MM')
),
comparado AS (
  SELECT ruta, mes, ingreso,
         LAG(ingreso) OVER (PARTITION BY ruta
                            ORDER BY mes) AS ingreso_anterior
  FROM   ingreso_mes
)
SELECT ruta,
       TO_CHAR(mes, 'YYYY-MM') AS mes,
       ingreso,
       ingreso_anterior,
       ROUND((ingreso - ingreso_anterior)
             / ingreso_anterior * 100, 2) AS variacion_pct
FROM   comparado
WHERE  ingreso_anterior IS NOT NULL
  AND  ingreso < ingreso_anterior * 0.7
ORDER BY ruta, mes;
```

Resultado:

```
RUTA      MES       INGRESO   INGRESO_ANTERIOR   VARIACION_PCT
BOG-BAQ   2026-04  1.000.000        3.600.000          -72,22
BOG-CLO   2026-04  1.200.000        3.600.000          -66,67
BOG-CTG   2026-03  2.500.000        4.000.000          -37,50
BOG-MDE   2026-03  3.000.000        4.800.000          -37,50
CLO-SMR   2026-04    800.000        2.500.000          -68,00
MDE-CTG   2026-04  1.200.000        1.900.000          -36,84
```

La consideración del enunciado

«Si una ruta no tuvo ventas en el mes anterior, esa fila no debe aparecer.» Esto se cumple gracias a que LAG devuelve nulo cuando no hay fila anterior dentro de la partición, y la condición ingreso_anterior IS NOT NULL descarta esas filas.

Compruébalo con la ruta CLO-SMR, que no tuvo operación en enero. Ejecuta la CTE comparado sin el filtro final y busca sus filas:

```
CLO-SMR  2026-02    900.000   (nulo)      <- primer mes con datos, se excluye
CLO-SMR  2026-03  2.500.000    900.000    <- subio, no aplica
CLO-SMR  2026-04    800.000  2.500.000    <- cayo 68 %, aparece
```

Además, el filtro IS NOT NULL cumple una segunda función: evita una división por nulo en el cálculo del porcentaje. Si el ingreso del mes anterior pudiera ser cero, habría que protegerse también con NULLIF(ingreso_anterior, 0).

Un caso límite que el enunciado no exige resolver, pero conviene entender

Mira la fila de MDE-CTG. Aparece en abril comparada contra 1.900.000, que es el ingreso de FEBRERO, no de marzo. La razón es que esa ruta no operó en marzo, y LAG devuelve la fila anterior que existe en los datos, no el mes calendario inmediatamente anterior.

Es decir: LAG trabaja sobre las filas que hay, no sobre los meses que deberían existir. Si el informe necesitara comparar siempre contra el mes calendario anterior, habría que generar previamente la serie completa de meses y cruzarla con los datos, de modo que los meses sin operación aparezcan explícitamente con ingreso cero o nulo.

Advertir este comportamiento por escrito era el criterio de nota máxima de esta pregunta.

Por qué no basta con GROUP BY

GROUP BY colapsa las filas de cada mes de forma independiente. Cada grupo se calcula aislado de los demás y el resultado no ofrece ningún mecanismo para que una fila consulte el valor de otra fila del mismo resultado. Las funciones de agregado responden preguntas del tipo «cuánto suma este grupo», no del tipo «cómo se compara este grupo con el anterior».

Las funciones analíticas existen precisamente para cubrir esa carencia: no colapsan las filas, sino que a cada fila le dan una ventana desde la cual observar a sus vecinas.

Cómo se resolvería sin funciones analíticas. Con un autojoin de la agregación mensual contra sí misma, emparejando cada mes con el anterior mediante ADD_MONTHS:

```
WITH ingreso_mes AS ( ... la misma CTE de antes ... )
SELECT a.ruta,
       TO_CHAR(a.mes, 'YYYY-MM') AS mes,
       a.ingreso,
       b.ingreso AS ingreso_anterior,
       ROUND((a.ingreso - b.ingreso) / b.ingreso * 100, 2) AS variacion_pct
FROM   ingreso_mes a
JOIN   ingreso_mes b
       ON  b.ruta = a.ruta
       AND b.mes  = ADD_MONTHS(a.mes, -1)
WHERE  a.ingreso < b.ingreso * 0.7
ORDER BY a.ruta, a.mes;
```

Funciona, y es la forma en que se resolvía esto antes de que existieran las funciones analíticas. Fíjate en dos diferencias respecto de la versión con LAG. Primero, el join interno excluye por sí solo los meses sin anterior, sin necesidad del filtro IS NOT NULL. Segundo, y más interesante: esta versión sí exige que el mes anterior sea el mes calendario anterior, de modo que MDE-CTG no aparecería en el resultado. Las dos consultas responden preguntas ligeramente distintas, y saber cuál corresponde al requerimiento es parte del trabajo.

Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte.

PARTE 3 — AUTOEVALUACIÓN

Preguntas de comprobación

Si puedes responder estas preguntas sin volver atrás, dominaste el material. Si alguna te cuesta, vuelve a la sección correspondiente.

- ¿En qué momento exacto del procesamiento de una sentencia se calculan las funciones analíticas, y qué dos consecuencias prácticas tiene ese momento?
- Una consulta con NOT IN devuelve cero filas y no produce error. ¿Cuál es la primera hipótesis que debes verificar, y con qué consulta la verificas?
- ¿Por qué unir una tabla con dos tablas hijas en la misma sentencia infla las sumas? ¿Cuántas filas produce un padre con 4 hijos de un tipo y 3 del otro?
- ¿Qué marco de ventana aplica Oracle cuando escribes ORDER BY dentro de OVER sin especificar marco? ¿En qué se diferencia de ROWS?
- Tienes RANK, DENSE_RANK y ROW_NUMBER. ¿Cuál usarías si necesitas exactamente tres filas por grupo aunque haya empates? ¿Y si los empates deben compartir puesto sin dejar huecos en la numeración?
- ¿Por qué ROWNUM <= 5 no devuelve las cinco mejores filas, pero FETCH FIRST 5 ROWS ONLY sí?
- LAG devuelve el mes anterior de los datos o el mes calendario anterior. ¿Cuál de los dos, y por qué importa la diferencia?

Los errores más frecuentes en este quiz

- **Afirmar que la consulta de la Pregunta 1 «se ejecuta pero da mal el resultado».** No se ejecuta. El ORA-00979 la detiene antes de producir nada.
- **Señalar RANK() OVER (ORDER BY SUM(...)) como defecto.** Es válido en Oracle y señalarlo revela el malentendido sobre el orden de evaluación.
- **Escribir «falta ROWS» sin explicar el porqué.** La corrección sin la explicación del marco RANGE y del tratamiento de los pares no demuestra comprensión, solo memoria.
- **Corregir el fan-out con COUNT(DISTINCT ...) y creer que el problema quedó resuelto.** Eso arregla el conteo, pero la suma de las tarifas sigue inflada. Las dos ramas deben agregarse por separado.
- **Usar RANK cuando el enunciado pide una numeración sin huecos, o ROW_NUMBER cuando pide que los empates compartan puesto.** Las tres funciones de ranking no son intercambiables: cada una responde a una regla de empate distinta.
- **Olvidar el desempate determinístico.** Una consulta que puede devolver resultados distintos en dos ejecuciones sobre los mismos datos no es una consulta terminada.

Las cinco ideas que deberías llevarte

```
1.  El orden en que SQL evalua las clausulas no es el orden en que se
    escriben. Casi todos los errores de este quiz salen de ahi.

2.  El nulo no es un valor: es la ausencia de valor. Cualquier operador
    que lo compare devuelve UNKNOWN, y UNKNOWN no pasa el filtro.

3.  Antes de agregar, revisa la cardinalidad de tus joins. Si dos ramas
    son de uno a muchos, el join las multiplica entre si.

4.  En funciones de ventana acumulativas, escribe siempre el marco de
    forma explicita y anade un criterio de desempate unico.

5.  Una consulta sintacticamente perfecta puede responder una pregunta
    distinta de la que se hizo. Relee el requerimiento al terminar.
```